

ПІДСУМКОВА МОДУЛЬНА РОБОТА
з дисципліни «Основи програмування мовою JavaScript»

Виконав:

Устич Максим

Група:

alk-43

Охоплення:

Теми 1–11

Форма контролю:

Комп'ютерне тестування

Київ 2026 р.

Частина 1. Тестові питання

Тема 1. Змінні і типи даних

Питання 1

Яке твердження про `const` є правильним?

- A) Змінну `const` можна перезаписати в будь-який момент*
- B) Змінну `const` не можна повторно присвоїти, але об'єкт за посиланням можна мутувати*
- C) `const` створює лише числові константи*
- D) `const` має функціональну область видимості*

Правильна відповідь:

B) Змінну `const` не можна повторно присвоїти, але об'єкт за посиланням можна мутувати.

Пояснення:

Ключове слово `const` не дозволяє повторно присвоювати значення змінній. Проте якщо змінна містить об'єкт або масив, їх внутрішні дані можна змінювати.

Приклад:

```
const user = {  
  name: "Max"  
};  
  
user.name = "Ivan";  
  
console.log(user.name);
```

Результат:

Ivan

Питання 2

Скільки існує примітивних типів даних у JavaScript?

- A) 3
- B) 5
- C) 7
- D) Тільки object

Правильна відповідь:

C) 7

Пояснення:

У JavaScript існує 7 примітивних типів даних:

1. string
2. number
3. bigint
4. boolean
5. undefined
6. symbol
7. null

Примітивні типи зберігають прості значення та не є об'єктами.

Приклад:

```
let text = "Hello";  
let number = 10;  
let isActive = true;  
  
console.log(typeof text);  
console.log(typeof number);  
console.log(typeof isActive);
```

Результат:

string

number

boolean

Питання 3

Що поверне вираз `"" || "default"` ?

- A) `""`
- B) `"default"`
- C) `true`
- D) `undefined`

Правильна відповідь:

B) `"default"`

Пояснення:

Оператор `||` повертає перше істинне значення.

Порожній рядок `""` у JavaScript вважається `false`, тому буде повернено `"default"`.

Приклад:

```
let result = "" || "default";
```

```
console.log(result);
```

Результат:

default

Питання 4

Який результат матиме вираз `0 == false` ?

- A) `true`
- B) `false`
- C) `undefined`
- D) `TypeError`

Правильна відповідь:

A) true

Пояснення:

Оператор `==` у JavaScript виконує нестроге порівняння з автоматичним приведенням типів.

Значення `0` і `false` після приведення типів вважаються рівними.

Приклад:

```
console.log(0 == false);
```

Результат:

true

Тема 3. Цикли

Питання 5

Який цикл найкраще підходить для перебору значень масиву без використання індексів?

A) for...in C) while

B) for...of D) do...while

Правильна відповідь:

B) for...of

Пояснення:

Цикл `for...of` використовується для перебору значень масиву без звернення до індексів.

Він послідовно повертає кожен елемент масиву.

Приклад:

```
const numbers = [10, 20, 30];

for (const number of numbers) {
  console.log(number);
}
```

Результат:

10

20

30

Питання 6

Що зробить оператор `continue` всередині циклу?

A) Повністю завершить цикл

B) Перейде до наступної ітерації, пропустивши решту коду в поточній

C) Перезапустить цикл з початку

D) Викличе помилку

Правильна відповідь:

B) Перейде до наступної ітерації, пропустивши решту коду в поточній.

Пояснення:

Оператор `continue` пропускає виконання коду, який залишився в поточній ітерації циклу, та переходить до наступної ітерації.

Приклад:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  
  console.log(i);  
}
```

Результат:

1
2
4
5

Тема 4. Функції

Питання 7

Чим arrow function відрізняється від звичайної функції щодо this ?

- A) Arrow function створює власний this*
- B) Arrow function не має власного this і бере його з зовнішнього контексту*
- C) Різниці немає*
- D) Arrow function забороняє використання this*

Правильна відповідь:

B) Arrow function не має власного this і бере його з зовнішнього контексту.

Пояснення:

Стрілочна функція (arrow function) не створює власний `this`.
Вона використовує значення `this` із зовнішньої області видимості.

Приклад:

```
const user = {  
  name: "Max",  
  
  regularFunction: function () {  
    console.log(this.name);  
  },  
  
  arrowFunction: () => {  
    console.log(this.name);  
  }  
};  
  
user.regularFunction();  
user.arrowFunction();
```

Результат:

Max

undefined

Питання 8

Що означає синтаксис `function sum(...numbers)` ?

- A) Функція приймає рівно один аргумент-масив*
- B) `numbers` збере всі передані аргументи у масив (rest parameters)*
- C) Функція не приймає аргументів*
- D) Це синтаксична помилка*

Правильна відповідь:

B) `numbers` збере всі передані аргументи у масив (rest parameters).

Пояснення:

Оператор ... називається `rest parameters`.

Він дозволяє функції приймати будь-яку кількість аргументів і автоматично об'єднує їх у масив.

Приклад:

```
function sum(...numbers) {  
  console.log(numbers);  
}
```

```
sum(1, 2, 3, 4);
```

Результат:

```
[1, 2, 3, 4]
```

Тема 5. Об'єкти

Питання 9

Що повертає `Object.entries({ a: 1, b: 2 })`?

- A) `["a", "b"]` C) `[["a", 1], ["b", 2]]`
B) `[1, 2]` D) `{ a: 1, b: 2 }`

Правильна відповідь:

C `[["a", 1], ["b", 2]]`

Пояснення:

Метод `Object.entries()` повертає масив пар [ключ, значення] для кожної властивості об'єкта.

Приклад:

```
const obj = {  
  a: 1,  
  b: 2  
};  
  
console.log(Object.entries(obj));
```

Результат:

```
[["a", 1], ["b", 2]]
```

Питання 10

Як за допомогою spread-оператора об'єднати об'єкти obj1 та obj2 у новий об'єкт?

- A) { obj1, obj2 } C) [...obj1, ...obj2]
B) { ...obj1, ...obj2 } D) obj1 + obj2

Правильна відповідь:

B) { ...obj1, ...obj2 }

Пояснення:

Spread-оператор ... дозволяє копіювати властивості одного об'єкта в інший.

Таким способом можна легко об'єднати декілька об'єктів у новий.

Приклад:

```
const obj1 = {  
  name: "Max"
```

```
};
```

```
const obj2 = {  
  age: 20  
};
```

```
const result = {  
  ...obj1,  
  ...obj2  
};
```

```
console.log(result);
```

Результат:

```
{ name: "Max", age: 20 }
```

Тема 6. Події та DOM

Питання 11

Що таке делегування подій?

- A) Видалення обробника після першого спрацювання*
- B) Встановлення одного обробника на батьківський елемент замість обробників на кожному дочірньому*
- C) Заборона стандартної поведінки браузера*
- D) Створення нової події вручну*

Правильна відповідь:

B) Встановлення одного обробника на батьківський елемент замість обробників на кожному дочірньому.

Пояснення:

Делегування подій — це спосіб обробки подій, коли один обробник встановлюється на батьківський елемент і реагує на події від дочірніх елементів.

Приклад:

```
const list = document.getElementById("list");

list.addEventListener("click", function(event) {
  if (event.target.tagName === "LI") {
    console.log(event.target.textContent);
  }
});
```

Результат:

При натисканні на елемент списку в консоль буде виведено текст вибраного елемента.

Питання 12

Який метод створює новий DOM-елемент?

- A) *document.createElement()* C) *document.appendChild()*
B) *document.newElement()* D) *document.querySelector()*

Правильна відповідь:

A) document.createElement()

Пояснення:

Метод `document.createElement()` створює новий HTML-елемент у DOM-дереві.

Після створення елемент можна додати на сторінку за допомогою інших методів.

Приклад:

```
const paragraph = document.createElement("p");  
  
paragraph.textContent = "Новий елемент";  
  
document.body.appendChild(paragraph);
```

Результат:

На сторінці буде створено новий абзац з текстом "Новий елемент".

Тема 7. Масиви

Питання 13

Що поверне `[1, 2, 3, 4].slice(1, 3)` ?

- A) `[1, 2]` C) `[2, 3, 4]`
B) `[2, 3]` D) `[1, 2, 3]`

Правильна відповідь:

B) `[2, 3]`

Пояснення:

Метод `slice(start, end)` повертає частину масиву від індексу `start` до `end`, не включаючи `end`.

Приклад:

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.slice(1, 3);  
  
console.log(result);
```

Результат:

[2, 3]

Питання 14

Який метод видаляє останній елемент масиву і повертає його?

- A) *shift()* C) *slice()*
B) *pop()* D) *splice()*

Правильна відповідь:

B) pop()

Пояснення:

Метод `pop()` видаляє останній елемент масиву та повертає його значення.

Приклад:

```
const numbers = [10, 20, 30];  
  
const removedElement = numbers.pop();  
  
console.log(removedElement);  
console.log(numbers);
```

Результат:

30

[10, 20]

Тема 8. Функції вищого порядку

Питання 15

Що поверне `[1, 2, 3, 4].filter(n => n % 2 === 0)` ?

- A) [1, 3] C) 6
B) [2, 4] D) true

Правильна відповідь:

B) [2, 4]

Пояснення:

Метод `filter()` створює новий масив з елементів, які відповідають умові.

У цьому прикладі залишаються лише парні числа.

Приклад:

```
const numbers = [1, 2, 3, 4];
```

```
const result = numbers.filter(n => n % 2 === 0);
```

```
console.log(result);
```

Результат:

[2, 4]

Питання 16

Який метод зводить масив до одного значення?

- A) `map()` C) `reduce()`
B) `forEach()` D) `find()`

Правильна відповідь:

C) reduce()

Пояснення:

Метод `reduce()` використовується для перетворення масиву в одне значення.

Наприклад, за допомогою `reduce()` можна знайти суму всіх елементів масиву.

Приклад:

```
const numbers = [1, 2, 3, 4];

const sum = numbers.reduce((total, number) => {
  return total + number;
}, 0);

console.log(sum);
```

Результат:

10

Тема 9. Замикання

Питання 17

Яке практичне застосування замикань (closures)?

- A) Прискорення циклів*
- B) Створення приватних змінних та інкапсуляція стану*
- C) Заміна класів*
- D) Сорткування масивів*

Правильна відповідь:

B) Створення приватних змінних та інкапсуляція стану.

Пояснення:

Замикання (closure) дозволяє функції запам'ятовувати змінні із зовнішньої області видимості навіть після завершення роботи зовнішньої функції.

Приклад:

```
function createCounter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
  
console.log(counter());  
console.log(counter());  
console.log(counter());
```

Результат:

1
2
3

Тема 10. Проміси та async/await

Питання 18

*Що повертає функція, оголошена з ключовим словом **async** ?*

- A) Звичайне значення C) **undefined***
*B) **Promise** D) **Callback***

Правильна відповідь:

B) Promise

Пояснення:

Функція, оголошена з ключовим словом **async**, завжди повертає **Promise**, навіть якщо всередині повертається звичайне значення.

Приклад:

```
async function getMessage() {  
  return "Hello";  
}
```

```
console.log(getMessage());
```

Результат:

```
Promise { "Hello" }
```

Питання 19

Для чого використовується `Promise.all(...)` ?

- A) Виконує проміси послідовно один за одним*
- B) Чекає завершення всіх промісів і повертає масив їхніх результатів*
- C) Повертає результат першого завершеного проміса*
- D) Скасовує всі проміси*

Правильна відповідь:

B) Чекає завершення всіх промісів і повертає масив їхніх результатів.

Пояснення:

Метод `Promise.all()` використовується для одночасного виконання кількох промісів.

Він завершується тоді, коли всі проміси успішно виконані, і повертає масив результатів.

Приклад:

```
const promise1 = Promise.resolve(10);  
const promise2 = Promise.resolve(20);  
  
Promise.all([promise1, promise2])  
  .then(result => {  
    console.log(result);  
  });
```

Результат:

[10, 20]

Тема 11. Класи та ООП

Питання 20

Що робить ключове слово `super` у конструкторі дочірнього класу?

- A) Створює статичний метод*
- B) Викликає конструктор батьківського класу*
- C) Робить поле приватним*
- D) Видаляє батьківський клас*

Правильна відповідь:

B) Викликає конструктор батьківського класу.

Пояснення:

Ключове слово `super` використовується в дочірньому класі для виклику конструктора батьківського класу та отримання його властивостей і методів.

Приклад:

```
class Animal {
    constructor(name) {
        this.name = name;
    }
}

class Dog extends Animal {
    constructor(name, breed) {
        super(name);

        this.breed = breed;
    }
}

const dog = new Dog("Rex", "Labrador");
```

```
console.log(dog.name);  
console.log(dog.breed);
```

Результат:

Rex

Labrador

Частина 2. Практичні завдання

Завдання 1

Напишіть функцію `summarizeNumbers(numbers)`, яка приймає масив чисел і повертає об'єкт зі статистикою:

- `count` — кількість чисел
- `sum` — сума всіх чисел
- `evenCount` — кількість парних чисел
- `max` — найбільше число
- `category` — рядок `"empty"`, якщо масив порожній; `"positive"`, якщо сума > 0 ; інакше `"non-positive"`

Використайте цикл для обчислень (без методів вищого порядку).

Приклад:

```
summarizeNumbers([4, 7, 2, 9]);  
// { count: 4, sum: 22, evenCount: 2, max: 9,  
   category: "positive" }  
  
summarizeNumbers([]);  
// { count: 0, sum: 0, evenCount: 0, max: undefined,  
   category: "empty" }
```

Код програми:

```

function summarizeNumbers(numbers) {
  let count = numbers.length;
  let sum = 0;
  let evenCount = 0;
  let max = undefined;

  if (count === 0) {
    return {
      count: 0,
      sum: 0,
      evenCount: 0,
      max: undefined,
      category: "empty"
    };
  }

  max = numbers[0];

  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];

    if (numbers[i] % 2 === 0) {
      evenCount++;
    }

    if (numbers[i] > max) {
      max = numbers[i];
    }
  }

  let category;

  if (sum > 0) {
    category = "positive";
  } else {
    category = "non-positive";
  }
}

```

```

return {
  count: count,
  sum: sum,
  evenCount: evenCount,
  max: max,
  category: category
};
}

```

```

console.log(summarizeNumbers([4, 7, 2, 9]));
console.log(summarizeNumbers([]));
console.log(summarizeNumbers([-5, 2, 1]));

```

Результат виконання:

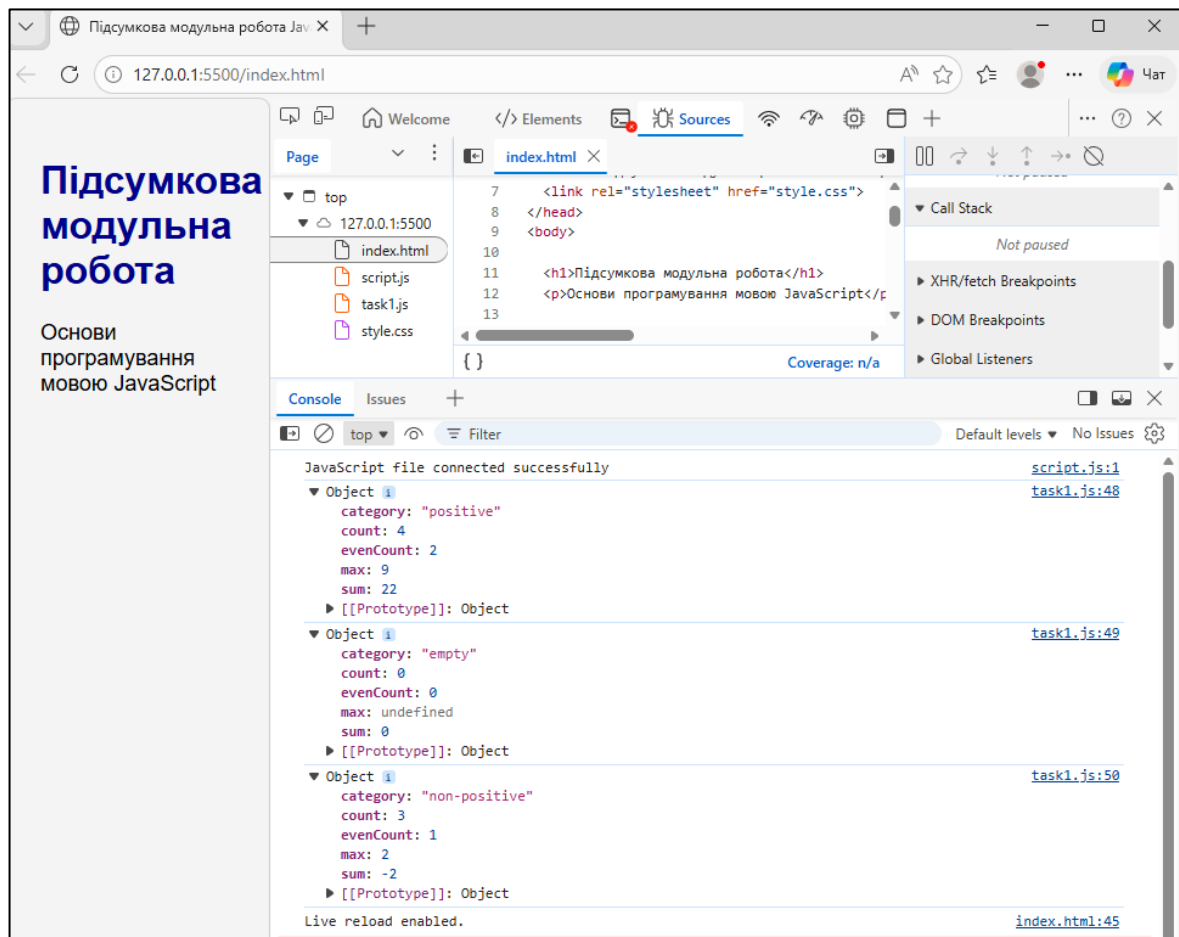


Рис. 1 – Результат роботи функції `summarizeNumbers()`

Пояснення

У функції `summarizeNumbers()` використовується цикл `for` для проходження по масиву чисел.

Під час виконання програми обчислюється сума елементів, кількість парних чисел та визначається найбільше число.

Якщо масив порожній, функція повертає категорію `"empty"`. Якщо сума чисел більша за нуль — `"positive"`, інакше — `"non-positive"`.

Завдання 2

Напишіть функцію `processProducts(products)`, яка приймає масив товарів і повертає об'єкт. Кожен товар: `{ name: "Чай", price: 50, inStock: true }`

Функція повинна повернути:

- `available` — масив назв товарів, що є в наявності (`inStock === true`)
- `totalPrice` — сума цін усіх товарів у наявності (використати `reduce`)
- `cheapest` — назва найдешевшого товару серед наявних
- `priceList` — масив рядків виду `"Чай – 50 грн"` для всіх товарів (використати `map`)

Приклад:

```
const products = [
  { name: "Чай", price: 50, inStock: true },
  { name: "Кава", price: 120, inStock: false },
  { name: "Цукор", price: 30, inStock: true }
];

processProducts(products);
// {
//   available: ["Чай", "Цукор"],
//   totalPrice: 80,
//   cheapest: "Цукор",
//   priceList: ["Чай – 50 грн", "Кава – 120 грн",
// "Цукор – 30 грн"]
// }
```

Код програми:

```
function processProducts(products) {
  const availableProducts = products.filter(product =>
    product.inStock === true);

  const available = availableProducts.map(product =>
    product.name);

  const totalPrice = availableProducts.reduce((sum, product)
    => {
      return sum + product.price;
    }, 0);

  let cheapest = null;

  if (availableProducts.length > 0) {
    cheapest = availableProducts.reduce((min, product) => {
      return product.price < min.price ? product : min;
    }).name;
  }

  const priceList = products.map(product => {
    return `${product.name} – ${product.price} грн`;
  });

  return {
    available: available,
    totalPrice: totalPrice,
    cheapest: cheapest,
    priceList: priceList
  };
}

const products = [
  { name: "Чай", price: 50, inStock: true },
  { name: "Кава", price: 120, inStock: false },
  { name: "Цукор", price: 30, inStock: true }
```

];

```
console.log(processProducts(products));
```

Результат виконання:

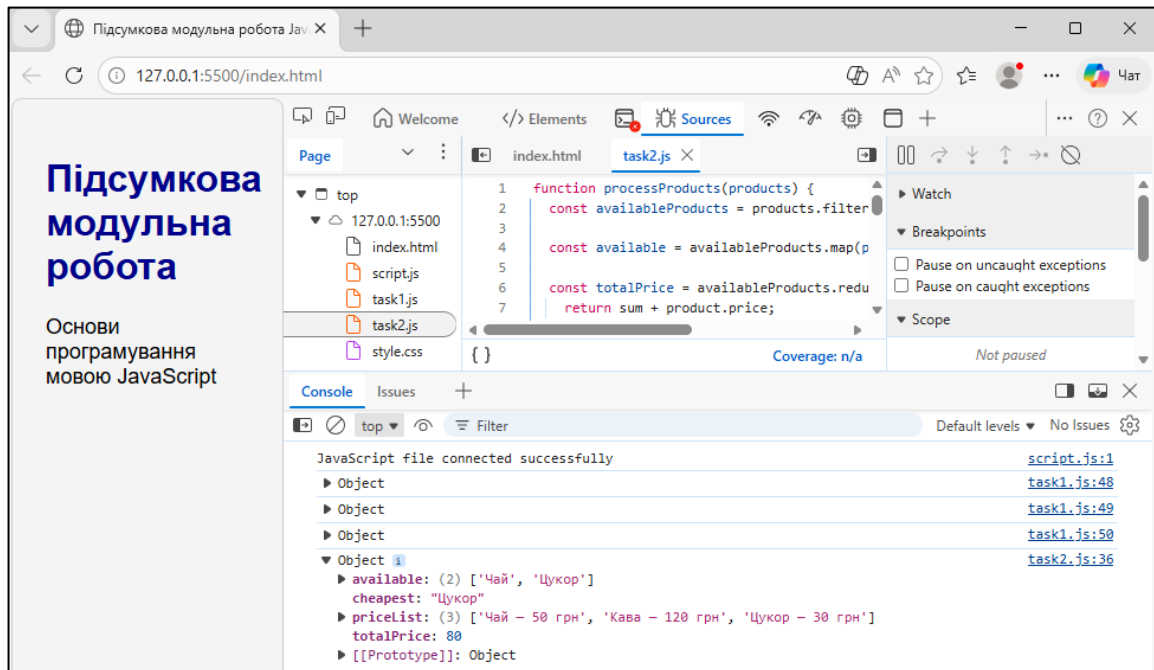


Рис. 2 – Результат виконання Завдання 2 у браузері

Пояснення

У програмі використовуються методи `filter()`, `map()` та `reduce()`.

Метод `filter()` використовується для відбору товарів, які є в наявності. Метод `map()` формує список назв товарів та список рядків з цінами. Метод `reduce()` обчислює загальну вартість товарів та допомагає знайти найдешевший товар.

Програма повертає об'єкт із інформацією про доступні товари, їхню загальну вартість та список цін.

Завдання 3

Реалізуйте функцію `createApiClient(baseUrl)`, яка використовує замикання для зберігання `baseUrl` та лічильника запитів. Вона повертає об'єкт з методами:

- `get(path)` — асинхронний метод, що виконує `fetch(baseUrl + path)`, повертає розпарсений JSON; у разі помилки повертає `{ error: "Запит не вдался" }`. Кожен виклик збільшує лічильник запитів.
- `getRequestCount()` — повертає кількість виконаних запитів.

Використайте `async/await`, `try...catch` та замикання для приватного лічильника.

Приклад:

```
const api =
  createApiClient("https://jsonplaceholder.typicode.com");
const user = await api.get("/users/1"); // об'єкт
користувача
const posts = await api.get("/posts"); // масив
постів
api.getRequestCount(); // 2
```

Код програми:

```
function createApiClient(baseUrl) {
  let requestCount = 0;

  return {
    async get(path) {
      requestCount++;

      try {
        const response = await fetch(baseUrl + path);

        if (!response.ok) {
          throw new Error("Помилка запиту");
        }
      }
    }
  };
}
```

```

}

const data = await response.json();
return data;
} catch (error) {
return {
error: "Запит не вдавсь"
};
}
},

getRequestCount() {
return requestCount;
}
};
}

const api =
createApiClient("https://jsonplaceholder.typicode.com");

async function testApiClient() {
const user = await api.get("/users/1");
console.log(user);

const posts = await api.get("/posts");
console.log(posts);

console.log("Кількість запитів:", api.getRequestCount());
}

testApiClient();

```

Результат виконання:

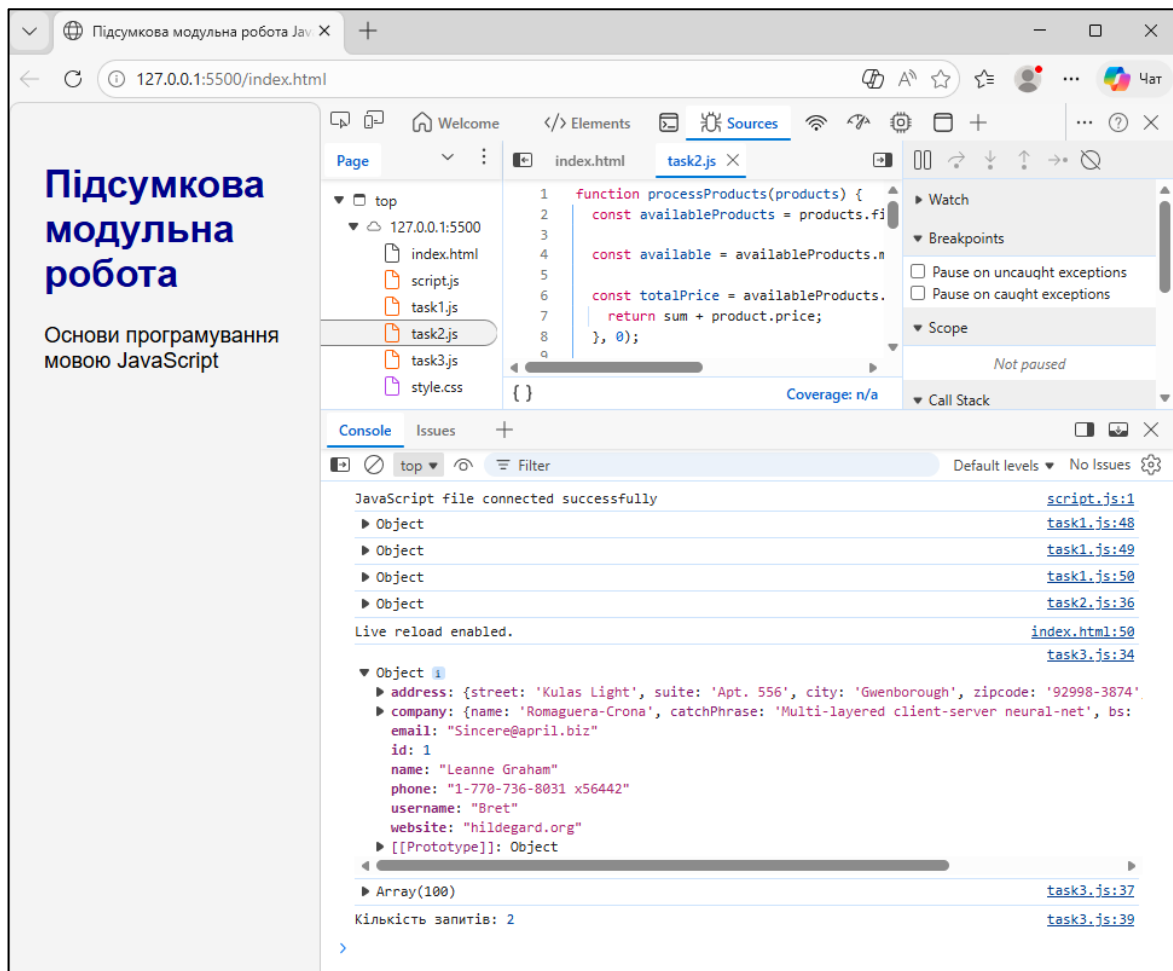


Рис. 3 – Результат виконання Завдання 3 у браузері

Пояснення

У програмі використовується `async/await` для виконання асинхронних HTTP-запитів до API.

Функція `fetch()` отримує дані з сервера у форматі JSON. Також реалізована обробка помилок через `try...catch`.

Програма підраховує кількість виконаних запитів та повертає отримані дані користувачів і постів.

Завдання 4

Створіть клас `ToDoList` для керування списком завдань та підключіть його до сторінки.

1. Клас `Task` з полями `id`, `text`, `done` (за замовчуванням `false`) та методом `toggle()`, що інвертує `done`.
2. Клас `ToDoList` з методами:
 - `add(text)` — створює нову `Task` з унікальним `id` і додає до списку
 - `remove(id)` — видаляє завдання за `id`
 - `getActive()` — повертає масив незавершених завдань
3. JavaScript-код, що при натисканні кнопки `#addBtn` бере текст з `<input id="taskInput">`, додає завдання через `ToDoList`, рендерить список у `<ul id="taskList">`, а кліком по елементу списку перемикає його стан `done` (використати делегування подій).

Приклад HTML:

```
<input id="taskInput" placeholder="Нове завдання">
<button id="addBtn">Додати</button>
<ul id="taskList"></ul>
```

Код програми:

```
class Task {
  constructor(id, text) {
    this.id = id;
    this.text = text;
    this.done = false;
  }

  toggle() {
    this.done = !this.done;
  }
}

class ToDoList {
  constructor() {
    this.tasks = [];
```

```

this.nextId = 1;
}

add(text) {
  const task = new Task(this.nextId, text);
  this.tasks.push(task);
  this.nextId++;
}

remove(id) {
  this.tasks = this.tasks.filter(task => task.id !== id);
}

getActive() {
  return this.tasks.filter(task => task.done === false);
}
}

const todoList = new TodoList();

const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");

function renderTasks() {
  taskList.innerHTML = "";

  todoList.tasks.forEach(task => {
    const li = document.createElement("li");
    li.textContent = task.done ? `${task.text} – виконано` :
    task.text;
    li.dataset.id = task.id;

    if (task.done) {
      li.style.textDecoration = "line-through";
    }

    taskList.appendChild(li);
  });
}

```

```

});

console.log("Усі завдання:", todoList.tasks);
console.log("Активні завдання:", todoList.getActive());
}

addBtn.addEventListener("click", function () {
const text = taskInput.value.trim();

if (text === "") {
alert("Введіть текст завдання");
return;
}

todoList.add(text);
taskInput.value = "";
renderTasks();
});

taskList.addEventListener("click", function (event) {
if (event.target.tagName === "LI") {
const id = Number(event.target.dataset.id);
const task = todoList.tasks.find(task => task.id === id);

if (task) {
task.toggle();
renderTasks();
}
}
});

todoList.add("Вивчити JavaScript");
todoList.add("Виконати модульну роботу");
renderTasks();

```

Результат виконання:

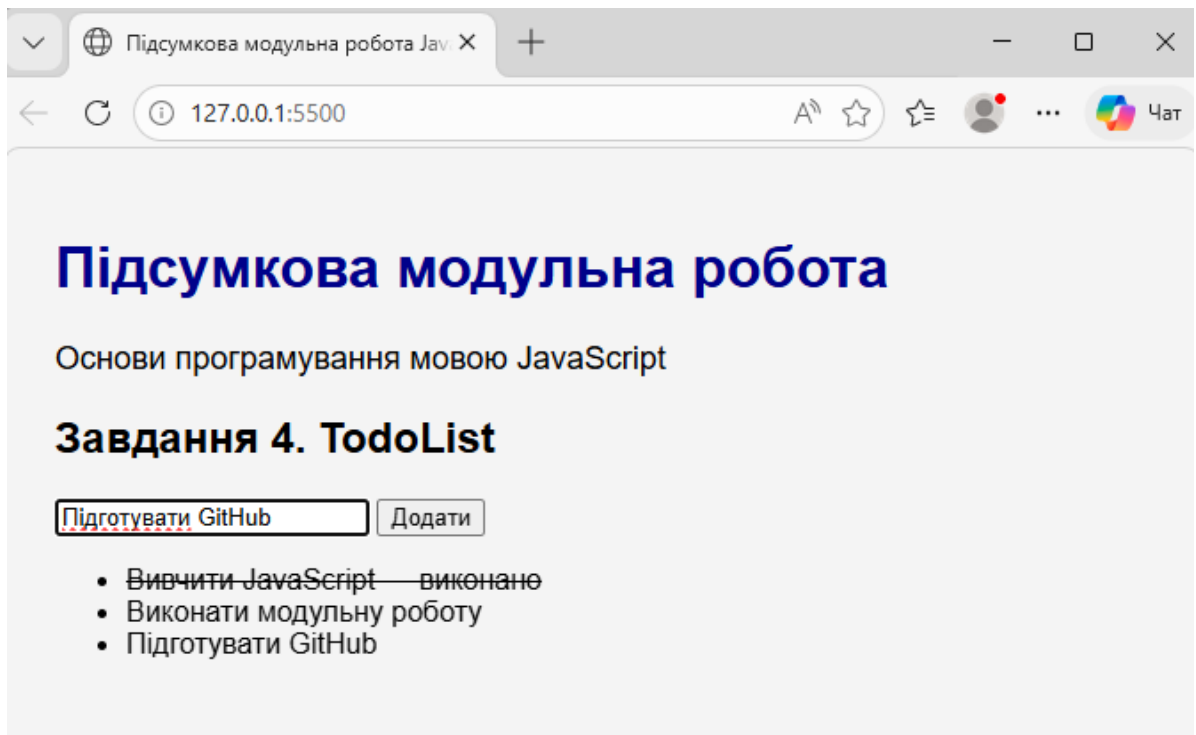


Рис. 4 – Результат виконання Завдання 4 у браузері

Пояснення

У програмі реалізовано клас `Task` для створення окремих завдань та клас `TodoList` для керування списком завдань.

Клас `Task` містить:

- ідентифікатор завдання `id`;
- текст завдання `text`;
- статус виконання `done`;
- метод `toggle()`, який змінює стан виконання завдання.

Клас `TodoList` дозволяє:

- додавати нові завдання;
- видаляти завдання;

- отримувати список активних завдань.

Для взаємодії зі сторінкою використовуються DOM-елементи:

- поле введення `<input>`;
- кнопка `<button>`;
- список `<ul>`.

При натисканні кнопки «Додати» нове завдання додається до списку та відображається на сторінці.

Також у програмі реалізовано делегування подій: при кліку на елемент списку змінюється статус виконання завдання, а виконані завдання перекреслюються.